

Procesamiento de tramas NMEA utilizando un microcontrolador ESP32-C3

Hugo J. Albarenque, Julián A. Barbero
Facultad de Ingeniería y Ciencias Hídricas de la UNL

juninhoalba@gmail.com, agustinbarbero16@gmail.com

Resumen— En el presente trabajo se desarrolló un sistema embebido utilizando el microcontrolador ESP32-C3 para la obtención y el procesamiento de los datos de posicionamiento geográfico provenientes del módulo GPS u-blox NEO-6M. El sistema permite al usuario seleccionar, por medio de comunicación por el puerto UART0, qué información desea obtener (coordenadas, altitud o velocidad), y por medio de UART1 recibir la información del GPS, esto permite mantener la interfaz de usuario (UART0) limpia y mostrarle directamente los datos depurados. Además, se indica el estado en el que se encuentra el procesamiento de la información mediante LEDs. El procedimiento consistió en implementarlo utilizando lenguaje C y ensamblador RISC-V para interactuar con los módulos (LEDs y GPS). Usando el entorno de desarrollo ESP-IDF (que simplifica la configuración de los registros de bajo nivel), se configuró la interfaz UART1. Esto permite comunicar al GPS correctamente sin contaminar la interfaz del usuario, ya que el puerto se utilizó para recibir las tramas NMEA con la información necesaria para la geolocalización. Luego de obtener la trama por medio de la memoria FIFO de la UART, se desarrolló un algoritmo de parsing que analizó carácter a carácter las cadenas recibidas y envió al usuario la información solicitada.

Palabras clave— UART, Parsing, Sensor GPS, Ensamblador RISC-V.

I. INTRODUCCIÓN

En el contexto actual del Internet de las Cosas (IoT) y los sistemas embebidos, la adquisición de datos de geolocalización es un requerimiento fundamental. Módulos receptores como el u-blox NEO-6M permiten obtener información precisa de posicionamiento global mediante la transmisión de tramas de texto que siguen un protocolo de la NMEA (National Marine Electronics Association). Para recibir esta información, los microcontroladores, como el ESP32-C3, disponen de periféricos de comunicación serial asíncrona (UART) que actúan como puente físico entre el módulo GPS y el microcontrolador.

Si bien los entornos de desarrollo oficiales, como ESP-IDF (Espressif IoT Development Framework), proveen controladores (drivers) de alto nivel que resuelven la lectura del puerto serie mediante interrupciones y buffers en memoria RAM, su uso exclusivo permite abstraerse de la interacción con los registros de bajo nivel. Depender únicamente de estas abstracciones oculta el comportamiento físico de la placa, impidiendo comprender cómo interactúa realmente el procesador RISC-V (Reduced Instruction Set Computer - V) con los registros mapeados en memoria y las memorias de tipo FIFO (First In, First Out) del hardware.

Por lo tanto, el objetivo del presente trabajo es diseñar e implementar un sistema híbrido de adquisición y procesamiento de datos GPS que evidencie el control sobre la arquitectura del microcontrolador y la capacidad de implementar un sistema embebido capaz de comunicar distintos periféricos. Para ello, se propone una arquitectura de software dividida en dos capas: una capa de muy bajo nivel desarrollada en Assembler RISC-V dedicada a la lectura de la FIFO física de la UART1 y a implementar rutinas para el manejo de los LEDs; y una capa de alto nivel desarrollada en lenguaje C, encargada de parsear las tramas NMEA y de ejecutar de forma secuencial las rutinas encargadas de prender y apagar los LEDs que sirven al usuario para entender en qué estado se encuentra el proceso de parsing.

II. MATERIALES Y MÉTODOS

Para el desarrollo del sistema de adquisición de datos se utilizó un microcontrolador ESP32-C3 como unidad central de procesamiento. La obtención de las coordenadas geográficas se realizó mediante un módulo comercial GPS u-blox NEO-6M, mientras que la visualización del proceso de *parsing* para el usuario se gestionó a través de una placa de LEDs. La comunicación entre ambos dispositivos se estableció mediante el periférico de comunicación serial asíncrona (UART1). Específicamente, el pin de transmisión (TX) del módulo GPS se conectó al puerto GPIO0 (General Purpose Input/Output) del ESP32-C3; el pin de recepción (RX) del sensor se omitió, ya que el diseño no requiere el envío de instrucciones hacia el GPS.

La alimentación del módulo satelital se suministró desde la salida de 5V de la placa base, aprovechando el regulador de voltaje integrado en el PCB (Printed Circuit Board) del sensor. En cuanto a la interfaz lumínica, se configuró mediante lógica inversa: todos los ánodos de la barra LED se conectaron al pin de 3.3V de la ESP32-C3, mientras que los cátodos de

los LEDs 1, 2 y 3 se vincularon a los puertos GPIO 1, GPIO 3 y GPIO 10, respectivamente. Esta disposición permite utilizar solo los pines de propósito general libres, sin tener que reasignar funciones, mientras que el encendido de cada indicador se produzca cuando el microcontrolador establece un estado lógico bajo (0V) en el pin correspondiente.

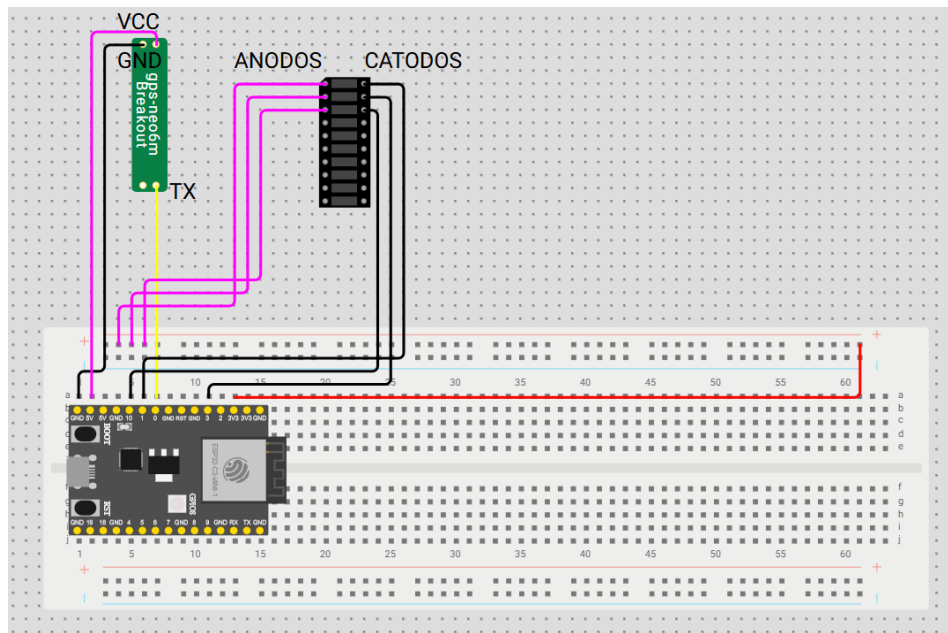


Fig. 1: Circuito descrito en el párrafo anterior

Antes de iniciar con el desarrollo de las rutinas principales, fue necesario habilitar y configurar el puerto UART1 empleado para la comunicación física con el módulo GPS. Para ello, se utilizó el entorno de desarrollo ESP-IDF, el cual provee una capa de abstracción inicial sobre la complejidad de los registros de bajo nivel.

Mediante la función `uart_driver_install(UART_NUM_1, 256, 0, 0, NULL, 0)`, se instalaron los controladores de software del puerto. En esta configuración, se asignó un tamaño de 256 bytes al búfer circular de recepción (RX) en memoria RAM —valor que la función requiere sea mayor al tamaño del buffer físico de la UART—, aunque el acceso a los datos se realiza directamente sobre la FIFO hardware, sin pasar por dicho buffer. El búfer de transmisión (TX) se estableció en 0 bytes, dado que el sistema opera de forma unidireccional y no transmite instrucciones hacia el sensor GPS.

Finalmente, los últimos tres parámetros (asociados a las colas de eventos y banderas de asignación) se configuraron en cero o nulos para deshabilitar la generación de eventos del sistema operativo ante la llegada de datos a la memoria FIFO. El sistema fue pensado para operar mediante consulta constante (polling), con estos argumentos se configura el puerto UART1 evitando que implemente interrupciones automáticas que interfieran con la lectura manual de los registros.

Posteriormente, para finalizar la parametrización del puerto, se definieron los parámetros lógicos de la transmisión. Se asignó una velocidad de 9600 baudios y se estableció el formato de trama 8N1 (8 bits de datos, sin bit de paridad y 1 bit de parada), cumpliendo estrictamente con las especificaciones del protocolo NMEA bajo el cual opera el módulo GPS. Además, se configuró la fuente de temporización del puerto, asignando el reloj de referencia para periféricos integrado en la arquitectura del microcontrolador. Por último, se deshabilitó el control de flujo por hardware, dado que el módulo u-blox NEO-6M carece de los pines físicos dedicados a dicha tarea. Para terminar de configurar el puerto de UART1, se le indica al entorno de desarrollo que el GPIO 0 será destinado a ser el pin RX del puerto y se envía de parámetro “-1” como pin TX de este puerto, indicando que no será utilizado.

Para la interacción directa con el hardware, se desarrollaron rutinas en lenguaje ensamblador que manipulan los registros específicos de la arquitectura RISC-V. Debido a la conexión física establecida, el control de la interfaz visual de LEDs requirió el acceso a los registros de configuración y salida: `GPIO_ENABLE_W1TS_REG` (para habilitar los pines), `GPIO_OUT_W1TS_REG` (para establecer el estado alto) y `GPIO_OUT_W1TC_REG` (para establecer el estado bajo o limpiar la salida).

Por otro lado, para gestionar la recepción de datos del módulo GPS, fue necesario operar sobre los registros de control del puerto serie: `UART1_STATUS_REG`, utilizado para verificar la disponibilidad de datos, y `UART1_FIFO`, desde donde se extraen los bytes recibidos. El acceso a la información contenida en estos registros se realizó mediante la aplicación de máscaras de bits, permitiendo aislar y manipular únicamente los bits de interés.

Con la declaración de estas direcciones de memoria y sus respectivas máscaras, fue posible implementar rutinas modulares capaces de extraer caracteres individuales de la cola de recepción (FIFO) de la UART1, así como funciones específicas para configurar los pines GPIO como salida y conmutar el estado (encendido/apagado) de los indicadores LED según el flujo del programa.

A partir de esto es posible utilizar estas rutinas para armar las cadenas correspondientes del GPS en código C y trabajarlas fácilmente, pero para poder mostrar la información solicitada por el usuario totalmente depurada es necesario conocer como están compuestas las tramas que envía el GPS.

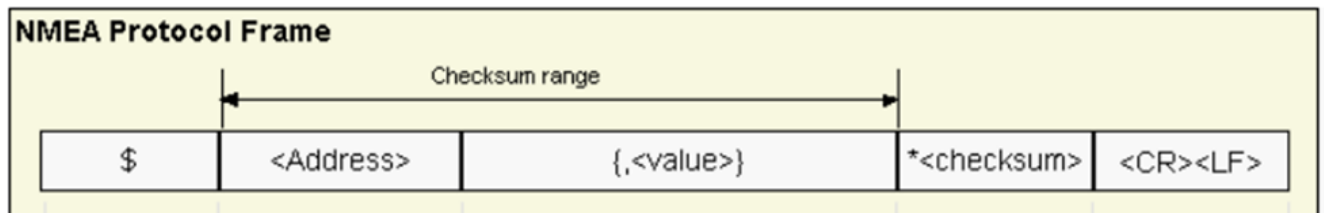


Fig. 2: Formato de las tramas enviadas por el GPS

Como se observa en la Fig. 2, las tramas del protocolo se estructuran mediante campos de encabezado (address) y datos (value). Para este proyecto, el filtrado se centró en las sentencias \$GPGGA (que provee latitud, longitud y altitud) y \$GPVTG (velocidad).

Aprovechando que los datos en el campo value se encuentran delimitados por comas, se implementó un algoritmo de iteración que contabiliza estos separadores hasta alcanzar el índice del parámetro deseado, extrayendo así la subcadena correspondiente. Posteriormente, se trabajó parte de la cadena (los grados) mediante la función estándar strtol, lo que facilita su manipulación aritmética y limpieza de formato (de esta manera no se muestran caracteres innecesarios como un cero a la izquierda).

Finalmente, se insertaron rutinas de retardo (delay) por software dentro del ciclo de procesamiento. Esta pausa artificial resulta necesaria para adaptar la alta velocidad de ejecución del microcontrolador a la percepción visual del usuario, permitiendo observar correctamente la secuencia de estados en los indicadores LED.

III. RESULTADOS

El sistema desarrollado permitió establecer una comunicación serial estable entre el microcontrolador ESP32-C3 y el módulo GPS u-blox NEO-6M a una velocidad de 9600 baudios. Se verificó la correcta recepción de las tramas NMEA, validando la detección de sentencias del tipo \$GPGGA y \$GPVTG.

El algoritmo de parsing implementado identificó el final de cada sentencia mediante la detección del carácter de salto de línea, procesando el flujo de bytes proveniente de la FIFO. A partir de ello, se extrajeron y aislaron los campos delimitados por comas correspondientes a latitud, longitud, sus respectivas orientaciones cardinales, altitud y velocidad. En el caso de las coordenadas geográficas, se logró la separación aritmética entre grados y minutos, asociando cada valor con su hemisferio correspondiente (N/S o E/W), lo que permitió su visualización en un formato legible para el usuario.

Como se evidencia en la Fig. 3, el sistema respondió adecuadamente a la interacción mediante la consola UART. Al seleccionar la opción de altitud, el sistema reportó un valor de 93.0 metros sobre el nivel del mar, y para las coordenadas, desplegó la ubicación precisa (Latitud S / Longitud W) en tiempo real. Paralelamente, los indicadores LED reflejaron fielmente las etapas del proceso (recepción de solicitud, búsqueda de datos y éxito), validando la sincronización entre las rutinas de ensamblador y la lógica de control en C.

```
=====
SISTEMA DE ADQUISICION DE DATOS GPS (UART1)
=====
Seleccione la informacion que desea consultar:
[1] Coordenadas (Latitud y Longitud)
[2] Altitud
[3] Velocidad
=====
>>> Opcion [1] recibida correctamente.
Obteniendo informacion del modulo GPS...
Aca esta la Longitud W: 60 grados y 30.35697 minutos
Aca esta la latitud S: 31 grados y 45.41592 minutos
Informacion obtenida correctamente

=====
SISTEMA DE ADQUISICION DE DATOS GPS (UART1)
=====
Seleccione la informacion que desea consultar:
[1] Coordenadas (Latitud y Longitud)
[2] Altitud
[3] Velocidad
=====
>>> Opcion [2] recibida correctamente.
Obteniendo informacion del modulo GPS...
Aca esta la altitud: 93.0 metros
Informacion obtenida correctamente
```

Fig. 3: Resultados obtenidos

IV. DISCUSIÓN Y CONCLUSIONES

En cuanto al modelo de adquisición implementado, se optó por un esquema de consulta constante (polling) sobre la FIFO del puerto UART1, deshabilitando explícitamente la generación de interrupciones automáticas. Esta decisión resulta adecuada para el contexto del presente trabajo, dado que el módulo u-blox NEO-6M transmite tramas NMEA a 9600 baudios, una velocidad lo suficientemente baja como para que el microcontrolador pueda atender la lectura sin riesgo de pérdida de datos. Sin embargo, esta estrategia presenta limitaciones de escalabilidad: si se incrementara la velocidad de transmisión, se incorporarían múltiples periféricos que demanden atención simultánea, o se requiriera que el procesador ejecute otras tareas en paralelo, el esquema de polling se convertiría en un cuello de botella. En ese escenario, un diseño basado en interrupciones resultaría más apropiado, permitiendo que el procesador atienda la llegada de datos solo cuando sea necesario y liberando ciclos de CPU para otras operaciones.

El desarrollo del presente trabajo demostró que, mediante la implementación de rutinas en lenguaje ensamblador, es posible gestionar eficientemente los recursos de un microcontrolador para la adquisición de datos de un módulo GPS, integrando simultáneamente una retroalimentación visual en tiempo real a través de indicadores LED.

En el contexto tecnológico actual, impulsado por el paradigma del Internet de las Cosas (IoT), este proyecto cobra especial relevancia al evidenciar la practicidad del entorno de desarrollo ESP-IDF. Se comprobó que dicho entorno facilita el diseño de sistemas embebidos complejos, permitiendo la coexistencia de una lógica de control en lenguaje C con la manipulación de bajo nivel necesaria para interactuar directamente con el hardware, prescindiendo de las abstracciones estándar cuando se requiere mayor control.

Como resultado, se obtuvo una interfaz de usuario depurada y funcional, que permite visualizar de manera transparente el procesamiento de las solicitudes y acceder a la información de geolocalización de forma directa, validando así la arquitectura híbrida propuesta.

REFERENCIAS

- [1] Espressif Systems, *ESP32-C3 Series Datasheet*, Versión 1.0. [En línea]. Disponible: https://documentation.espressif.com/esp32-c3_datasheet_en.pdf
- [2] Espressif Systems, *ESP32-C3 Technical Reference Manual*. [En línea]. Disponible: https://documentation.espressif.com/esp32-c3_technical_reference_manual_en.pdf
- [3] Espressif Systems, *ESP-IDF Programming Guide*. [En línea]. Disponible: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>
- [4] Espressif Systems, "Universal Asynchronous Receiver/Transmitter (UART) API Reference", en *ESP-IDF Programming Guide*. [En línea]. Disponible: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html#_CPPv411uart_port_t
- [5] u-blox, *u-blox 6 Receiver Description Including Protocol Specification* (Documento No. GPS.G6-SW-10018). [En línea]. Disponible: https://content.u-blox.com/sites/default/files/products/documents/u-blox6_ReceiverDescrProtSpec_%28GPS.G6-SW-10018%29_Public.pdf